SCA Backend: CRUD, Encryption & Firestore Guide

How each create / read / update / delete request is processed, what is encrypted, and where every record is stored.

Project: **sca-be** (Node 20, TypeScript, Express 5, Zod) · Database: **Firebase Firestore** · File storage: **Cloudflare R2** · Updated: 30 October 2026

1. How a request is processed
2. How encryption works
3. Firestore collections map
4. CRUD procedures by module
5. Cloudflare R2 storage
6. Encrypted data summary
7. Plain-text fields to review
8. Rate limits

ENCRYPTED AES-256-GCM, can be decrypted by the server

HASHED one-way, cannot be decrypted

PLAIN stored as normal text

1. How a request is processed

Every request goes through the same pipeline before it touches the database. Code lives in a feature folder: `src/modules/<feature>/<feature>.{routes, controller, service, validator, types}.ts`.

1. **Security layer** (`app.ts`)
   Helmet security headers, CORS that only accepts the frontend (`CLIENT_URL`), and a JSON body limit of 100 KB.

2. **Rate limit per IP** (`rateLimit.middleware.ts`)
   Max 300 requests per minute per IP address. Over the limit → `429`.

3. **Authentication** (`auth.middleware.ts → requireAuth`)
   Reads `Authorization: Bearer <JWT>`, verifies it and sets `req.user.id`. Missing or invalid → `401`. Only `/auth/*` routes are public.

4. **Rate limit per user**
   400 requests per 10 minutes per signed-in user (plus stricter limits on login, forgot password and uploads; see section 8).

5. **Role check** (`role.middleware.ts → requireRole("dev")`, developer routes only)
   Reads the user's `accessRoles` from Firestore on every request, so role changes work right away. No matching role → `403`.

6. **Validation** (`validate.middleware.ts` + `*.validator.ts`)
   Zod checks and cleans the body, query and URL params (trims text, enforces lengths, formats, required fields). Invalid → `400` with field errors. Files are checked by type and size (images 10 MB, documents 50 MB).

7. **Controller** (`*.controller.ts`)
   Takes the user id and the validated input and calls the service. No business logic here.

8 **Service** (`*.service.ts`)
Business logic. Builds the path `users/{userId}/{collection}` so a user can only reach their own records (another user's id → `404`). **Encrypts sensitive fields before writing** and **decrypts them after reading**.

9 **Storage**
Records go to Firestore; file bytes are encrypted and go to Cloudflare R2.

10 **Audit log** (`utils/audit.ts`)
For students, cases, appointments and files, writes `{ action, entity, entityId, createdAt }` to `users/{userId}/auditLogs`. Never the record's content.

11 **Response**
Success: `{ success: true, data, pagination? }`. Error: `{ success: false, message, details? }`. Create → `201`, delete → `204`. Raw server errors are never sent to the browser.

Lists: every list is paginated (max 15 per page) and newest first. Filters Firestore can do run in the query; text search and filters on encrypted fields run on the server after decrypting, then the result is paged.

2. How encryption works

Method	Algorithm	How it is used
ENCRYPT text / JSON	AES-256-GCM (`utils/encryption.ts`)	Sensitive fields are encrypted before saving and decrypted only for the signed-in owner. Stored as one string: `iv:authTag:ciphertext` (base64). A new random IV is used every time, so the same value never looks the same twice. Groups of fields are encrypted together as JSON (`encryptJson`).
ENCRYPT files	AES-256-GCM (binary)	File bytes are encrypted before uploading to R2: `[12-byte IV][16-byte authTag][ciphertext]` . Downloaded files are decrypted on the server and streamed to the browser.
HASH passwords	bcrypt, 12 rounds	Passwords are never stored or decrypted; login compares the typed password with the hash.
HASH email-link tokens	SHA-256	Only the hash is saved as the `authTokens` document id. The raw token exists only inside the email link.
HASH lookups	HMAC-SHA256 with the encryption key	`phoneHash` : finds duplicate mobile numbers without decrypting phones.

Encryption key: `ENCRYPTION_KEY` (64 hex characters) in `.env` / Render environment. If it is lost or changed, every encrypted field and uploaded file becomes unreadable. Back it up securely and never commit it.

3. Firestore collections map

Personal records live **under their owner**, so ownership is enforced by the path itself. Shared data (support reports, system updates, tokens) is top-level.

```
Firestore
├── users/{userId}                      account + profile (one per person)
│   ├── students/{id}                   student records          → data ENCRYPTED
│   ├── cases/{id}                      case records             → data ENCRYPTED
│   ├── appointments/{id}               appointments             → data ENCRYPTED
│   ├── files/{id}                      uploaded file info       → data ENCRYPTED
│   │                                   (file bytes ENCRYPTED in R2)
│   ├── categories/{id}                 Settings dropdown lists  → data ENCRYPTED
│   ├── todo/board                      To Do board              → data ENCRYPTED
│   └── auditLogs/{id}                  who created/updated/deleted/downloaded what
│
├── supportRequests/{id}                bug reports + feature requests (incl. votes)
│                                       → media ENCRYPTED in R2
├── updates/{id}                        published system updates → media ENCRYPTED in R2
└── authTokens/{sha256(token)}          email links: verify email, reset password,
                                        developer invitation (id is HASHED)
```

users/{userId} fields

Field	Protection	Notes
`name`, `email`	PLAIN	Email must be plain because login looks it up. Unique per account.
`password`	HASHED	bcrypt. `null` for Google-only accounts.
`phone`	ENCRYPTED	Normalized to `09XXXXXXXXX` before encrypting.
`phoneHash`	HASHED	HMAC-SHA256, used to block duplicate mobile numbers.
`address`	ENCRYPTED	JSON: region, province, city, barangay (PSGC `{ code, name }`) and street.
`accessRoles`	PLAIN	e.g. `["sca-1"]`, `["sca-1","dev"]`. Older accounts may have a single `accessRole`.
`role`, `school`	PLAIN	Job title and school details (name, ID, region, division, year, month, head, privacy).
`photo`	PLAIN info	`{ storageKey, mimeType, updatedAt }`. The picture itself is encrypted in R2.
`firebaseUid`, `emailVerifiedAt`, `readUpdateIds`, `readNotificationIds`, `createdAt`	PLAIN	Google link, confirmation time, read notifications (newest 300 kept).

4. CRUD procedures by module

Every record a user owns (students, cases, appointments, files, categories, To Do board) is stored as `{ referenceNumber, data, createdAt, updatedAt }`. `data` is the whole record encrypted as one JSON string. Only the reference number and times stay plain (for sorting). Search and filters run on the server after decrypting.

4.1 Accounts & authentication → `users`, `authTokens`

CRUD	Endpoint	Procedure
Create	`POST /auth/register`	Validate name, email, password (8–72), PH mobile, address → in a **transaction**: reject if the email or phoneHash already exists → hash password (bcrypt), encrypt phone and address, create `users/{id}` with role `sca-1` and the default Settings categories → save SHA-256 of a one-time token in `authTokens` and email the confirmation link (Brevo).
Create	`POST /auth/google`	Verify the Firebase ID token → find the user by Firebase UID or email (link it), or create a new account → return our JWT.
Read	`POST /auth/login`, `GET /auth/me`	Find by email, compare bcrypt hash, require a confirmed email → return JWT + profile (phone and address decrypted).
Update	`POST /auth/verify-email`, `/reset-password`	Hash the token from the link, find it in `authTokens`, check type and expiry, delete it (one use) → set `emailVerifiedAt` or a new password hash.
Create	`POST /auth/register-developer`	From an invitation link: new email → verified account with role `dev`; existing account → `dev` is added to its roles instead (no duplicate account).

4.2 Profile → `users/{userId}` + R2

CRUD	Endpoint	Procedure
Read	`GET /profile`	Read the user document → decrypt phone and address → return `{ user, school }` (never the password hash).
Update	`PUT /profile`	Validate → reject if another account has the same email or phoneHash (`409`) → encrypt phone and address → update the document.
Create / Update	`POST /profile/photo`	Check type (PNG/JPG/WEBP) and size (10 MB) → encrypt bytes → upload to R2 key `{userId}/profile-photo` (replaces the old one) → save photo info on the user.
Read	`GET /profile/photo`	Download from R2 → decrypt → send the image.
Delete	`DELETE /profile/photo`	Delete the R2 object → set `photo: null`.

4.3 Students → `users/{userId}/students`

CRUD	Endpoint	Procedure
Create	`POST /students`	Validate (student name and date encoded required) → **encrypt the whole form as `data`** → add with a generated `recordId` → audit log.
Read	`GET /students`, `/students/:id`	Load newest first → decrypt → name search, date range and dropdown filters (sex, grade & section, …) run after decrypting → paginate.
Update	`PUT /students/:id`	Check it exists under this user (else `404`) → re-encrypt → **replace** the document → audit log.
Delete	`DELETE /students/:id`	Check it exists → delete → audit log.

PLAIN recordId, createdAt, updatedAt only.
ENCRYPTED `data`: the whole student form (name, date encoded, sex, grade & section, school year, birthday, LRN, address, contacts, guardians, interests, help requested, parent concerns, review, remarks, …).

4.4 Cases → `users/{userId}/cases`

CRUD	Endpoint	Procedure
Create	`POST /cases`	Validate (intake date, learner name, follow-up date required) → **encrypt the whole case as `data`** → add with a generated `caseNumber` → audit log.
Read	`GET /cases`, `/cases/:id`	Load newest first → decrypt → search by learner name, intake date range and status (overdue / today / scheduled / accomplished) after decrypting → paginate.
Update	`PUT /cases/:id`	Check it exists → re-encrypt → **replace** the document (older plain-text cases lose their plain fields) → audit log.
Delete	`DELETE /cases/:id`	Check it exists → delete → audit log.

PLAIN caseNumber, createdAt, updatedAt only.
ENCRYPTED `data`: learner name, grade & section, concern, action, referred to, classification, intake date, follow-up date, status, remarks.

4.5 Appointments → `users/{userId}/appointments`

CRUD	Endpoint	Procedure
Create	`POST /appointments`	Validate (student, date, time required) → **encrypt the whole appointment as `data`** → add with a generated `appointmentId` → audit log.
Read	`GET /appointments`, `/:id`	Load newest first → decrypt → search, dates, status and staff run after decrypting → paginate.
Update	`PUT /appointments/:id`	Check it exists → re-encrypt → **replace** the document → audit log.
Delete	`DELETE /appointments/:id`	Check it exists → delete → audit log.

PLAIN appointmentId, createdAt, updatedAt only. ENCRYPTED `data`: student, staff, date, time, duration, type, reason, status, reminder, rescheduled from, notes.

4.6 Files → `users/{userId}/files` + R2

CRUD	Endpoint	Procedure
Create	`POST /files`	Check type (pdf, xls, xlsx, csv, ppt, pptx, png, jpg) and size (documents 50 MB, images 10 MB) → encrypt bytes → upload to R2 under a **random key** `{userId}/{uuid}` (the file name is never in the path) → save name, size, type and key **encrypted as** `data` → audit log.
Read	`GET /files`, `/files/:id/download`	List info (search by name, filter by kind). Download: read from R2 → decrypt → send → audit log (`download`).
Delete	`DELETE /files/:id`	Delete the R2 object → delete the record → audit log.

PLAIN createdAt only. ENCRYPTED `data` (name, type, size, storage key) and the file content (in R2).

4.7 Settings categories → `users/{userId}/categories`

CRUD	Endpoint	Procedure
Create	`POST /categories`	Decrypt the user's categories and reject a duplicate name (`409`) → add `{ name, options: [] }` **encrypted as** `data` → clear the cache.
Read	`GET /categories`	Served from a short in-memory cache per user, otherwise read newest first and paginate.
Update	`PATCH /categories/:id`, `/:id/options`	Decrypt → rename (no duplicates) or replace the options list → re-encrypt and save → clear the cache.
Delete	`DELETE /categories/:id`	Delete → clear the cache.

PLAIN createdAt only. ENCRYPTED `data`: name and options.

4.8 To Do board → `users/{userId}/todo/board`

CRUD	Endpoint	Procedure
Read	`GET /todo/board`	Read the single board document and decrypt it; new users get To Do / In Progress / Done.
Create / Update / Delete	`PUT /todo/board`	The whole board is saved at once (columns and cards) **encrypted as** `data`. Every card's column must exist; max 30 columns and 500 cards.

ENCRYPTED `data`: all columns and cards.

4.9 Support reports & feature requests → `supportRequests` (top-level) + R2

CRUD	Endpoint	Procedure
Create	`POST /support`	Any user. Validate type (bug / feature), subject, description, severity → encrypt up to 6 images/videos and upload to R2 under `support/{reportId}/{uuid}` → save the report with the reporter's name and email. Feature requests start as *under review*.
Read	`GET /support/board`, `/support/:id`, `/features`	Developers see the Kanban board; everyone sees feature requests (reporter hidden, only `mine`). Media is decrypted on download.
Update	`PATCH /support/:id`	Developers only: status (todo / in-progress / done), assignee (must be on the team), review (under review / approved / rejected).
Update	`POST /features/:id/vote`	In a **transaction**: add or remove the user in `voterIds` and update `votes` (one vote per user, approved requests only).

PLAIN type, subject, description, severity, status, review, assigneeId, reporter, voterIds, votes, dates. ENCRYPTED attachments (in R2).

4.10 System updates → `updates` (top-level) + R2

CRUD	Endpoint	Procedure
Create	`POST /updates`	Developers only, for a done report. Sanitize the editor HTML (removes scripts and unsafe tags) → encrypt media and upload under `updates/{updateId}/{uuid}` → save → notify every other user in real time.
Read	`GET /updates`, `/updates/:id`	Everyone. Paginated, newest first. Media decrypted on download.
Update	`POST /updates/:id/read`, `POST /notifications/read`	Saves read ids on the user (`readUpdateIds`, `readNotificationIds`).

PLAIN title, contentHtml (sanitized), authorName, dates. ENCRYPTED media (in R2).

4.11 Team → `users` (roles)

CRUD	Endpoint	Procedure
Read	`GET /team`	Developers only. Users with the `dev` role (new and older role fields), each listed once.
Create / Update	`POST /team/invitations`	Existing account → add `dev` to its `accessRoles`. New email → save a hashed invitation token in `authTokens` (72 hours) and email the link.

4.12 Reports & notifications (read only)

`GET /reports/dashboard`, `/reports/summary`, `/reports/notifications` read `cases` and `appointments`, decrypt them on the server and return **counts only**. Remarks and notes are never included. `GET /notifications/stream` is a real-time connection (Server-Sent Events) and stores nothing.

5. Cloudflare R2 storage

Every file is encrypted on the server before upload; R2 never receives readable content. Only the key is stored in Firestore, and keys are never sent to the browser.

R2 key	Linked from	Content
`{userId}/{uuid}`	`users/{id}/files/{id}.storageKey`	Uploaded documents and images **ENCRYPTED**
`{userId}/profile-photo`	`users/{id}.photo.storageKey`	Profile picture **ENCRYPTED**
`support/{reportId}/{uuid}`	`supportRequests/{id}.attachments[]`	Bug report images/videos **ENCRYPTED**
`updates/{updateId}/{uuid}`	`updates/{id}.media[]`	System update images/videos **ENCRYPTED**

6. Encrypted data summary

Where	Field	Protection	Reason
`users/{id}`	password	**bcrypt**	Login credential
`users/{id}`	phone	**AES-256-GCM**	Personal contact info
`users/{id}`	phoneHash	**HMAC-SHA256**	Duplicate check without decrypting
`users/{id}`	address (region, province, city, barangay, street)	**AES-256-GCM**	Home address
`users/{id}/students/{id}`	data (the whole student record)	**AES-256-GCM**	Student personal record
`users/{id}/cases/{id}`	data (the whole case)	**AES-256-GCM**	Confidential case record
`users/{id}/appointments/{id}`	data (the whole appointment)	**AES-256-GCM**	Confidential appointment
`users/{id}/files/{id}`	data (name, type, size, storage key)	**AES-256-GCM**	File names can include student names
`users/{id}/categories/{id}`	data (name, options)	**AES-256-GCM**	Dropdown values (staff names, sections)
`users/{id}/todo/board`	data (columns, cards)	**AES-256-GCM**	Cards can mention students
`authTokens/{id}`	document id (the link token)	**SHA-256**	Email confirmation, password reset, developer invitation
Cloudflare R2	all files, photos and media	**AES-256-GCM**	May contain student records

Records saved before full encryption are still readable and can be converted once with `npm run db:encrypt-all` (add `-- --dry-run` to test first). Deploy the backend before running it.

7. Plain-text fields to review

These are not encrypted, usually because Firestore needs to search or filter them:

Field	Why it is plain
`users.email`, `users.name`	Login looks users up by email. Could use an email hash if it must be encrypted.
`users/{id}/auditLogs`	Only the action, record type, record id and time (no content).
`supportRequests` subject / description / reporter	Developers read them on the board. Users should not put student details in support messages.
`updates.contentHtml`	Shown to every user (sanitized HTML).

8. Rate limits

Over a limit, the API returns `429` with a "please wait" message and a `Retry-After` header.

Applies to	Limit	Counted per
Every API request	300 per minute	IP address
Any request after sign-in (create, read, update, delete)	400 per 10 minutes	user
Sign-in (password or Google)	3 failed attempts per 10 minutes	IP address
Forgot password	2 per 3 minutes (first + resend)	IP address
Register, resend confirmation	5 per hour	IP address
Email links (verify, reset, developer invite)	20 per 15 minutes	IP address
File uploads (Files page and profile photo)	6 per 10 minutes	user
Contact Support / feature requests	5 per 10 minutes	user
Developer invitations	20 per hour	user